MyBud Student Assistant

Technical project report

Bikash Kushwaha

MyBud brings five common student tasks into one web application: conversational help, code review, recipe suggestions, travel planning, and expense tracking. This portfolio edition repairs the supplied application, makes its operating modes explicit, and adds persistent expense workflows.

Project outcome

The application runs locally with Django and SQLite. The assistant tools provide disclosed sample responses, with local Python syntax parsing in the code tool. Optional live Gemini requests use a configurable model and a server-side API key. The expense tracker stores entries and a monthly budget for the current browser session.

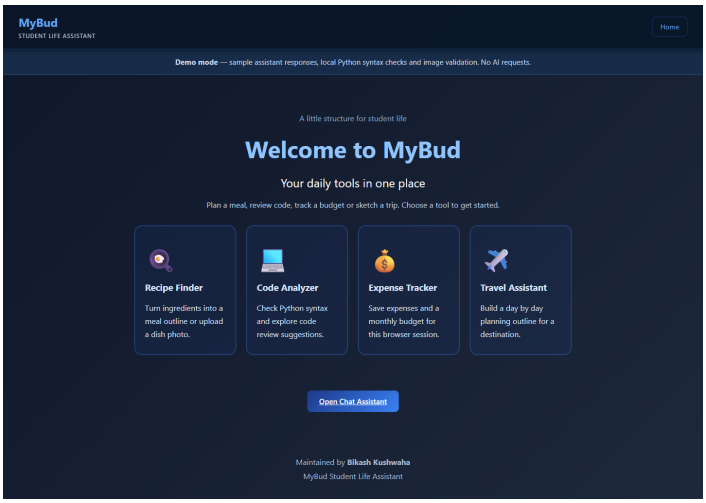


Figure 1. The running dashboard links to the five tools using the preserved dark blue interface.

Area	Scope
Assistant tools	Chat, code review, recipe suggestions with optional image input, and a structured travel itinerary.
Expense tools	Add and delete entries, reset the month, and review NPR spending against a monthly budget.
Evidence boundary	Local automated checks and browser verification. Sample responses do not measure AI quality.

The project is maintained by Bikash Kushwaha and adapted from supplied starter code. The acknowledgement file records that provenance without claiming sole original authorship.

Architecture and request flow

MyBud uses a server-rendered Django application with JavaScript enhancements for tool requests and interface updates. The browser provides task input; Django validates it, calls an assistant service or a database workflow, and returns a response for display. Credentials remain on the server.

Layer	Responsibility
Templates and static files	Render the dashboard and tool pages; collect input; present loading, result, and error states.
Routes and views	Map URLs to page handlers or API handlers, validate submitted data, enforce request rules, and scope expense access.
Assistant services	Provide a single configuration boundary for disclosed canned samples or optional live Gemini responses.
Django sessions	Associate the current browser with its expense records and monthly budget.
SQLite models	Persist expenses and budget data for the local demonstration.

Assistant request sequence

A user opens a tool page and submits the task fields. The view validates required values and limits before dispatching to the configured provider. The response includes the tool output and sufficient mode information for the interface to identify a sample response. Invalid input and provider failures produce readable errors rather than a server traceback in the page.

Expense request sequence

Django establishes a session key for the browser. Each expense query is restricted to that key and the current month. A validated create writes the record; deleting a record removes it only from the same session. The page reloads or refreshes the returned summary to show the stored monthly totals and budget balance.

Application safeguards

The portfolio repairs include CSRF protection for mutating browser requests, validation before provider or database work, and safer display of returned text. Environment-driven settings separate local development from deployment configuration. These measures support the documented local use case; a production launch still needs deployment-specific review.

Source references: project/settings.py, project/urls.py, main/urls.py, main/views.py, main/models.py, main/services/, templates/, static/.

Assistant tools and provider modes

The assistant tools share a provider boundary so that the application can be demonstrated offline and configured for a live model without changing the user-facing workflow. The selected mode is disclosed in the interface.

Mode	Behavior and boundary
Sample mode	Default local mode. Returns disclosed demonstration content without model inference or an API key. The code tool also parses Python syntax. This mode demonstrates local behavior without evaluating AI answer quality.
Live Gemini mode	Uses google-genai and the configured model. Requires valid credentials and provider availability. Output quality, latency, quota, and costs depend on that external service.

Tool	User workflow
Chat	Submit a question or study prompt and read the response. Sample mode labels its example content.
Code review	Submit a code snippet and read review suggestions. Demo mode parses Python syntax and returns fixed review tips. This tool does not execute submitted code.
Recipes	Submit ingredients and optionally an image. The request validates the upload before processing and returns recipe suggestions.
Travel	Provide a destination and trip duration. Read the returned day-by-day itinerary.

Image handling

The image endpoint decodes and validates JPEG, PNG, or WebP bytes with Pillow. It rejects invalid content, files over 5 MB, and images over 16 million pixels. Accepted images are converted to RGB, reduced to a maximum 1600 by 1600 bounding box, and re-encoded as JPEG before a possible live request. Demo mode reports that the image was validated and does not claim food recognition.

Response handling

Input, returned text, and errors are rendered as user-visible content. The interface must not treat arbitrary returned strings as trusted executable markup. Loading and error states let the user understand whether a request is in progress, complete, or rejected.

Evaluation boundary

The verification record confirms the documented local flows. No benchmark score, response accuracy rate, or live-provider performance claim is inferred from sample output. Travel and recipe suggestions should be checked for current facts and individual suitability before use.

Source references: main/views.py, main/services/, .env.example, main/tests.py, docs/VERIFICATION.md.

Expense data and ownership

The expense tracker provides real local persistence independently of the assistant provider. A record remains available after a page reload because it is stored in SQLite. Ownership follows the Django browser session rather than an authenticated account.

Data	Purpose
Expense	session_id, month, category, amount, and created_at. Amounts have two decimal places and must be positive.
Budget	session_id, month, amount, and last_set. A unique constraint permits one budget per session and month; zero is allowed.
Monthly summary	Current month in Asia/Kathmandu, NPR total, budget, remaining amount, exceeded flag, entries, and category totals.

Validation and operations

The service converts submitted values to Decimal, rejects non-finite values, negative numbers, boolean values, missing values, values at or above 100000000, and amounts with more than two decimal places. Expense amounts must be greater than zero; the budget may be zero. Valid values are quantized to two decimal places, and summaries use exact database aggregation.

Adding saves the category and amount under the current session and month, assigned using Asia/Kathmandu time. Deleting checks ownership before removing a record. Reset removes that session's expenses and budget for the month. Entries can be replaced through delete and add.

Budget behavior

Setting the budget creates or updates the current month's amount in NPR. The interface compares recorded spending with that amount and calculates the remaining balance. A negative balance means recorded expenses exceed the budget.

The captured example records NPR 800.25 spent against NPR 1,000.00, leaving NPR 199.75. These entries verify the workflow and do not report personal finances.

Session ownership and storage

A separate browser session cannot access another session's expense records. Losing the session cookie can remove access to prior entries. This edition has no account recovery or cross-device synchronization. SQLite provides local persistence; database and session state remain outside version control. Hosted use needs a plan for durable storage, backups, account ownership, and data lifecycle rules.

Source references: main/models.py, main/migrations/, main/views.py, main/tests.py, .gitignore.

Interface evidence and interaction design

The frontend retains the project's dark blue visual direction. Tool pages use comparable input and result areas so that the workflow remains familiar as users switch tasks. The screenshots below were captured from the running local application.

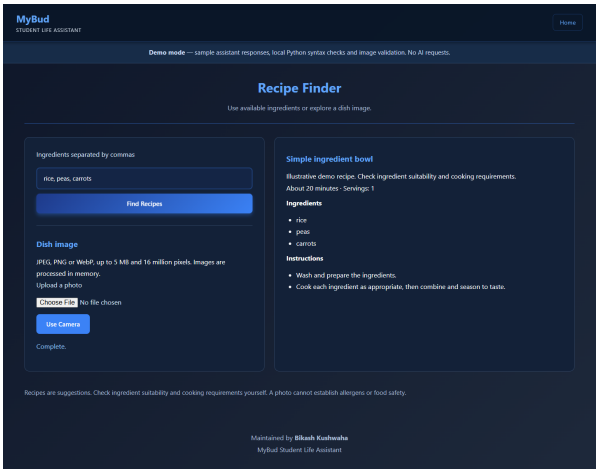


Figure 2. The recipe workflow accepts ingredients and an optional image. The displayed output identifies the active sample mode.

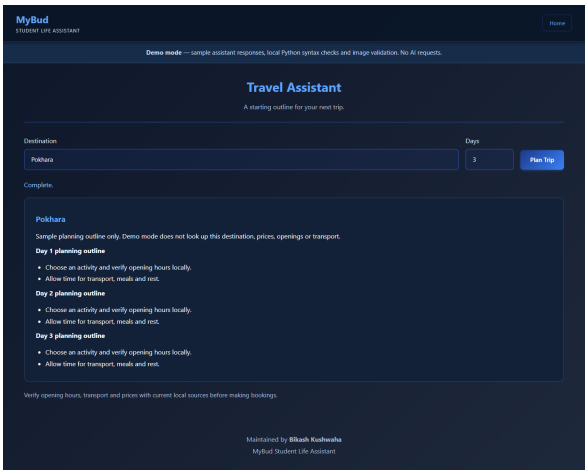


Figure 3. The travel tool organizes its result as an itinerary after destination and duration input.

Readable state changes

A submission communicates that work is in progress. A successful request presents a visible result; an invalid request presents a readable error. The interface preserves user input long enough to correct a rejected value, and the tool pages retain navigation back to the dashboard.

Expense interaction evidence

The expense workflow was checked with records and a budget, including data persistence after reload and the delete and reset paths. The summary reflects the current month in NPR. Browser-session isolation is verified separately from the visual design.

Source references: docs/screenshots/, templates/, static/, docs/VERIFICATION.md.

Verification and observed results

The local verification run passed 23 backend tests. Django reported no system-check issues, the migration drift check passed, and Ruff lint and format checks passed. Browser checks exercised the six rendered routes and the implemented tool workflows. These outcomes refer to the documented portfolio edition and its local demo configuration.

Area	Recorded evidence
Backend tests	23 passing automated tests
Django checks	No system-check issues; no migration drift
Code quality	Ruff lint and format checks passed
Browser flows	All tools, uploads, expenses and session isolation
Responsive checks	Six routes at desktop and mobile widths

Behavior covered by the checks

The page and API checks cover route rendering, displayed identity and mode, valid response contracts, malformed JSON, input bounds, disallowed HTTP methods, and CSRF enforcement.

Expense checks cover exact Decimal totals, invalid monetary values, budget handling, deletion, reset, and isolation between two browser sessions. Image checks cover valid uploads, malformed content, and a corrupt PNG checksum returning a validation error.

Provider-boundary tests use mocks for live success and failure, missing configuration, invalid schemas, private-error sanitization, and response-envelope overwrite rejection. Submitted code is parsed for Python syntax without execution.

Manual browser verification

The browser run exercised chat, code analysis, recipe generation, a real image upload, travel itineraries, expense persistence and deletion, and second-session expense isolation. An injected chat-markup check confirmed safe text display. All six routes were checked at 1440 by 1000 and 390 by 844 viewport sizes with no page overflow, console errors, or failed requests. Camera hardware and authenticated live Gemini calls were not tested.

What these results establish

The recorded results support the documented local application behavior and the tested validation boundaries. They do not establish live AI accuracy, production security, concurrent-user capacity, accessibility conformance, or success in an untested hosting environment. The exact commands and outcomes are retained in docs/VERIFICATION.md for repeatable review.

Source references: docs/VERIFICATION.md, main/tests.py, docs/screenshots/.

Local setup and configuration

With Python 3.12, create an environment, install the verified lockfile, migrate, and collect static assets. Run these Windows PowerShell commands from the project directory. The README contains the full setup sequence.

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements-lock.txt
.venv\Scripts\python manage.py migrate
.venv\Scripts\python manage.py collectstatic --noinput
.venv\Scripts\python manage.py runserver
```

Provider configuration

Setting	Purpose
MYBUD_MODE	demo for disclosed local samples; live for Gemini requests.
GEMINI_API_KEY and GEMINI_MODEL	Required together in live mode. Select a model supported by the configured provider account.
DJANGO_DEBUG and DJANGO_SECRET_KEY	Development switch and server secret. Production requires an explicit secret.
DJANGO_ALLOWED_HOSTS	Explicit hostnames allowed to reach the application.
Other deployment settings	DJANGO_CSRF_TRUSTED_ORIGINS, DJANGO_TRUST_PROXY, and optional DJANGO_DB_PATH configure origins, proxy trust, and database storage.

Copy .env.example to .env; Django loads it through python-dotenv. Keep keys outside version control. Live requests send task input to Gemini with a 30-second timeout. Returned JSON must pass structure validation before a successful result is displayed.

SDK reference: Google Gen AI SDK documentation.

Verification commands

```
.venv\Scripts\python manage.py check
.venv\Scripts\python manage.py test
.venv\Scripts\python -m ruff check .
.venv\Scripts\python -m ruff format --check .
```

Runtime data

Database migrations create the local schema. Expense records, budgets, and sessions are stored in the local SQLite database. Keep the virtual environment, environment secrets, database, caches, and temporary uploads out of the repository. A fresh clone initializes its own runtime data.

Deployment reference: Django 5.2 deployment checklist.

Source references: README.md, requirements-lock.txt, .env.example, project/settings.py, .gitignore.

Limitations provenance and future work

Current limitations

The offline assistant responses are canned samples, so they demonstrate the interface and application flow without evaluating a language model. Live mode requires external credentials and depends on provider availability. The expense tracker uses browser sessions instead of user accounts, and SQLite is configured for local use. The project does not include payment processing, booking, code execution, or guaranteed advice.

Future development

Priority	Proposed work
Ownership and recovery	Introduce account authentication, explicit data ownership, recovery, and an account data deletion flow.
Live assistant evaluation	Record a reviewed prompt set, provider/model configuration, output checks, and failure handling using live credentials.
Expense portability	Add a reviewed export format, backup strategy, and durable hosting database when deployment requirements are known.
Deployment and usability	Test the chosen production setup, add observability and rate controls, and evaluate the interface with representative users.

Source acknowledgement

This portfolio edition is maintained by Bikash Kushwaha and is adapted from supplied starter code. Maintenance work includes repaired routes and tool flows, a configurable assistant provider, persistent expense operations, input validation, configuration cleanup, and verification materials. The edition does not claim sole original authorship of all starter assets or implementation.

Project evidence references

Reference	Use in this report
README.md and .env.example	Setup, operating modes, and configuration.
main/ and project/	Implemented behavior, models, tests, routing, and settings.
docs/VERIFICATION.md	Recorded checks and test outcomes.
docs/screenshots/	Captured evidence from the running application.
Source acknowledgement	SOURCE_ACKNOWLEDGEMENTS.md records the supplied starter-code provenance.